第 6 章　低地球轨道卫星通信仿真

本章重点阐述低地球轨道卫星通信的发展和典型卫星系统，如 Iridium 和 Teledesic，并利用 NS-2 仿真分析上述卫星系统的数据传输性能，旨在让读者了解低地球轨道卫星通信系统应用现状和分析方法。

6.1 预备知识

6.1.1 低轨卫星网络发展[1]

低轨卫星组网是一种利用运行轨道高度较低的卫星建立的通信网络，如图 6.1 所示。一种特殊类型的低轨道是极轨道，其具有高倾角(接近 90 度)，因而卫星绕着两极移动。Iridium 和 Teledesic 是极轨卫星系统的例子，后续将针对它们进行分析。低轨卫星既可通过星间链路进行组网，也可同高轨卫星混合组网，形成多层空间网络。根据组网方式不同，卫星网络可划分为 3 类，即无星间链路卫星网络、有星间链路卫星网络和动态星间链路卫星网络。其中，无星间链路卫星网络基于地面设施进行组网，如全球星系统；有星间链路卫星网络通过星间链路进行连接且连接方式固定，不进行动态调整，如铱星系统；动态星间链路卫星网络的星间连接方式不固定，会随网络拓扑和业务变化而变化。

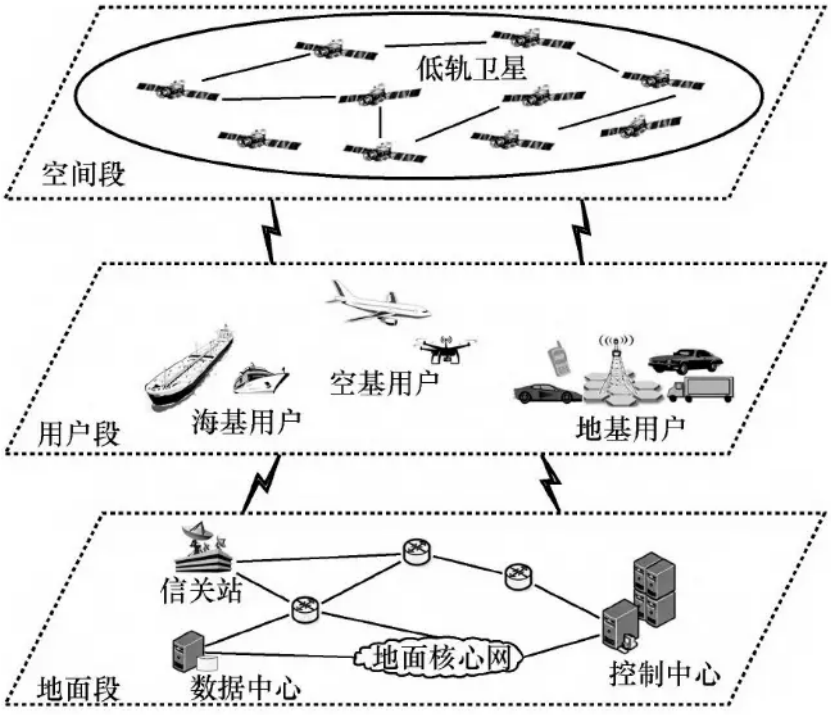


图 6.1　低轨卫星网络体系架构示意图

[1] https://spacefan.github.io/2019/02/12/LEOComm/

国外发展：2015 年，Space X 推出“星链(Starlink)”计划，拟发射 4.2 万颗低轨卫星，实现覆盖全球的高速互联网接入服务。2021 年，火箭制造商 Astra 拟部署 1.3 万颗低轨卫星，支持通信、环境和自然资源等应用。2022 年，亚马逊公司推出“柯伊伯项目”计划，拟部署 3236 颗低轨卫星组成太空卫星网络，为全球提供高速宽带互联网接入服务。目前，One Web 拥有世界第 2 大规模的卫星星座，在轨卫星数量已达 540 颗，仅次于 Starlink。2016 年，Telesat 宣布推出 Lightspeed 计划，拟发射 1671 颗低轨卫星，提供全球网络服务。位于卢森堡的 Leo Sat 公司推出 Leo Sat 星座，利用激光通信的方式部署 108 颗低轨卫星，以实现高质量的数据服务通信网络，链路速率最大能达 10Gbit/s。俄罗斯 SPUTNIX 公司计划在 2025 年完成部署约 200 颗低轨卫星。澳大利亚 Fleet 公司计划发射 100 颗纳米卫星，主要服务于物联网。2021 年 6 月韩国政府表示在 10 年内建设 100 颗微小卫星组成的卫星星座。

国内发展：“十四五”规划和“2035 年远景目标”纲要明确提出要建设高速泛在、天地一体、集成互联、安全高效的卫星互联网产业。为此，中国航天科技集团和中国航天科工集团分别制定了面向低轨卫星组网的“鸿雁星座”和“虹云工程”计划，其中“鸿雁星座”由 300 颗低轨道卫星及全球数据业务处理中心组成，“虹云工程”由 156 颗低轨卫星构成。北京国电高科打造和运营我国首个低轨卫星物联网星座，即“天启星座”，计划部署 38 颗低轨卫星。航天行云推出“行云系统”，拟发射 80 颗低轨道小卫星，建设一个覆盖全球的天基物联网。电子科技大学参与研制的“TFSTAR 号”卫星搭载 AI 处理能力，助力星载能力提升。中科院软件所启动基于软件定义卫星技术的“天智工程”，可用于低轨互联网建设。此外，还有民营企业推动的“银河 Galaxy”计划，拟发射 2800 颗低轨互联网卫星。

6.1.2 低轨卫星组网特点

低轨卫星网络是通过发射一定规模的卫星进行组网构建具备实时信号处理的空间通信系统，是一种能够向地面及空中终端提供接入等通信服务的新型网络，主要包括以下几个特点：

①**网络可靠性高且灵活**：组网方式灵活，单颗卫星发生故障后易进行网络切换，且不受自然灾害影响，大部分时间内低轨卫星网络可提供稳定且可靠的通信服务；

②**时延低**：通信链路均为视距，传输时延和路径损耗相对较小且稳定，能够支持视频通话、网络直播、在线游戏等实时性要求较高的应用；

③**容量大**：卫星数量较多且通常采用 Ka/V 频段或更高频段，可实现超过 500Mbit/s 大容量通信，支持海量终端接入需求；

④**地面网络依赖性弱**：星上处理技术的实现，可通过星间链路提供全球通信服务，而不需

要全球部署地面信号站，摆脱对地面基础设施的依赖；

⑤**多种技术协同发展**：多种相关技术协同应用，如点波束、多址接入、频率复用等技术，可缓解低轨卫星网络中存在的频率资源紧张等问题；

⑥**可实现全球覆盖**：多颗卫星协同组网，可实现全球无缝覆盖，不受地域限制，能够将网络扩展到远洋、沙漠等信息盲区。

建设新一代低轨卫星网络将是建设 6G 空间互联网的重要一部分，是实现全球互联的核心解决方案。

6.1.3 Iridium 星座

“铱星”星座是美国摩托罗拉公司于 1987 年提出的一种利用低轨道星座实现全球个人卫星移动通信的系统，它与现有通信网相结合，可实现全球数字化个人通信。“铱星”系统具有星间通信链路，能够不依赖地面转接为地球上任意位置的终端提供服务。“铱星”星座构型为玫瑰星座，卫星均匀部署在南北方向 677Km 高的 6 条极轨近圆轨道上，倾角为 86.4 度。每颗卫星载有 3 个 16 波束相控阵天线，投射的多波束在地球表面形成 48 个蜂窝区。每颗卫星拥有 4 条 Ka 频段的星间通信链路，两条用于建立同轨道面前后方向卫星的星间链路，星间距离 4021~4042Km；另两条用于建立相邻轨道面间卫星的通信链路，星间距离 2700~4400Km。异轨道面间链路的天线可根据加载到卫星上的星历信息进行指向调整，波束宽度足以适用纬度控制和卫星位置保持的容差。卫星在轨重量 320Kg，工作寿命 5~8 年。图 6.2 所示为“铱星”星座三维构型示意图。



图 6.2 “铱星”星座三维构型示意图

6.1.4 Teledesic 星座

Teledesic 也称“空中因特网”系统，由微软公司和麦考蜂窝通信公司共同提出的一个庞大

计划。该系统旨在建立一个覆盖全球的宽带卫星通信网络，提供双向、交互式服务。Teledesic 系统最初计划由 840 颗卫星组成，均匀分布在 21 个低地球轨道上，轨道倾角约为 98.2 度，每个轨道平面至少有 40 颗工作卫星。之后该设计被简化，改由 288 颗卫星分布在 12 个轨道，轨道高度 1375 公里。卫星每 99 分钟绕地球一周，覆盖地球表面的 95%，与关口站之间的链路速率可达 155Mbit/s。

Teledesic 用户终端包括固定终端、移动终端、SCADA 终端和团体终端等，能够提供包括宽带全数字双向交换业务在内的多种服务，如语音、数据、视像和交互式多媒体传输。系统采用 Ka 频段，上行频率为 28.6～29.1GHz，下行频率为 18.8～19.3GHz。多址方面，Teledesic 系统采用多种技术的组合，包括空分多址、时分多址和频分多址等。然而，由于电信业泡沫的破灭，Teledesic 在 2002 年被紧急叫停。尽管如此，Teledesic 系统的理念和技术对全球卫星通信网络的发展产生了深远的影响。

表 6.1　Iridium 和 Teledesic 系统对比

	Iridium	**Teledesic**
海拔高度	780km	1375km
轨道面	6	12
每个轨道面卫星	11	24
倾斜角度	86.4°	84.7°
轨道面间隔	31.6°	15°
缝隙间隔	22°	15°
卫星仰角	8.2°	40°
面内定相	yes	yes
面间定相	yes	no
每颗卫星星间链路数	4	8
星间链路带宽	25Mbps	155Mbps
上行/下行链路带宽	1.5Mbps	1.5Mbps
交叉缝星间链路	no	yes
星间链路纬度阈值	60°	60°

6.2 实验环境

本实验采用官方的 NS-2 仿真器源码，无需进行模块扩展，卫星网络模块可在路径“ns-2.35/satellite”中找到。实验所用系统为 Linux Ubuntu 20.04 且运行在 VirtualBox 6.1.50 软件中。具体环境安装详见导引实验。

为便于分析和表现实验效果，实验采用 CBR 作为数据源，并利用 UDP 进行数据传输。结果演示上，通过分析 trace 文件来对比时延的变化和通信距离的关系(发送和接收节点设置不同地点的纬度和经度坐标)，而实际数据传输路径中卫星节点个数不同，也会造成端到端的时延

的不同。有关纬度和经度获取可访问链接：https://api.map.baidu.com/lbsapi/getpoint/index.html，打开的界面如下图所示。

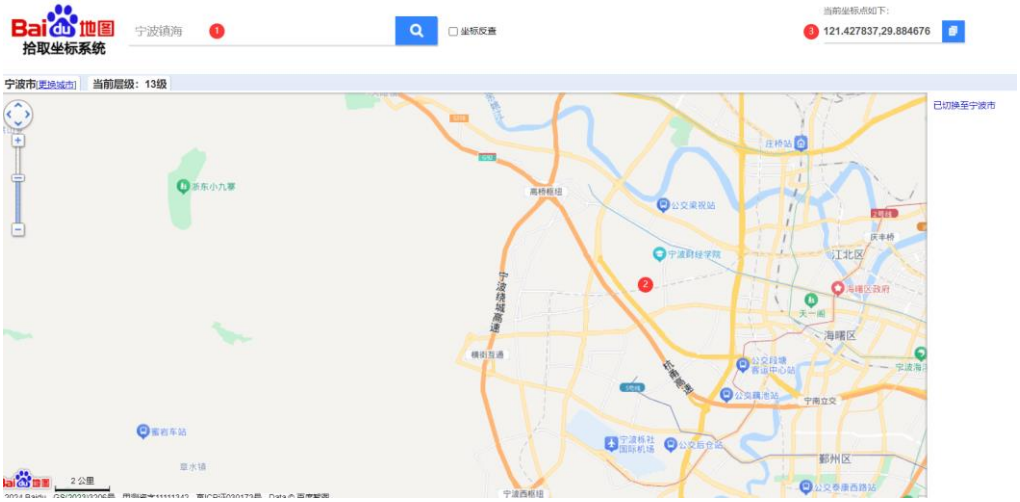


图 6.3　纬度和经度获取

具体获取纬度和经度信息步骤（上图编号）：在①处输入需要查询的经纬度的城市名并点击“ ”搜索（如“宁波镇海”），然后用鼠标在②所示的地图上选定详细位置，最后在③所示位置查看纬度和经度信息。

6.3 实验步骤

6.3.1 Iridium 仿真

该实验构建了包含 86 个节点、6 个轨道面的星座拓扑，同一个轨道面和跨轨道面的卫星链路设置如下代码片段所示(完整代码见“**ns-2.35/tcl/ex/sat-iridium-links.tcl**”)。

```
#以下代码设置轨道面 1 的带宽、接口队列和队列长度，其他轨道面相同
$ns add-isl intraplane $n0 $n1 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n1 $n2 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n2 $n3 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n3 $n4 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n4 $n5 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n5 $n6 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n6 $n7 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n7 $n8 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n8 $n9 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n9 $n10 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl intraplane $n10 $n0 $opt(bw_isl) $opt(ifq) $opt(qlim)
#以下代码设置轨道面 1 和轨道面 2 间节点的链路带宽、接口队列和队列长度，其他轨道面间的设置类似
$ns add-isl interplane $n0 $n15 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n1 $n16 $opt(bw_isl) $opt(ifq) $opt(qlim)
```

```
$ns add-isl interplane $n2 $n17 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n3 $n18 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n4 $n19 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n5 $n20 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n6 $n21 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n7 $n22 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n8 $n23 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n9 $n24 $opt(bw_isl) $opt(ifq) $opt(qlim)
$ns add-isl interplane $n10 $n25 $opt(bw_isl) $opt(ifq) $opt(qlim)
```

卫星节点生成如下代码片段所示(完整代码见“**ns-2.35/tcl/ex/sat-iridium-nodes.tcl**”)。共 6 个轨道面，每个轨道 11 个节点，每个节点的位置和轨道面设置如下。相邻轨道面的间隔是 31.6°，缝隙处是 22°，非均匀。每个轨道面 11 颗星，所以 360/11=32.73°。

```
#设置第 1 个轨道面，plane 为轨道面编号，其他轨道上的节点设置类似
set plane 1;#轨道编号设置
set n0 [$ns node]; $n0 set-position $alt $inc 0 0 $plane;#设置第 1 轨道节点位置
set n1 [$ns node]; $n1 set-position $alt $inc 0 32.73 $plane
set n2 [$ns node]; $n2 set-position $alt $inc 0 65.45 $plane
set n3 [$ns node]; $n3 set-position $alt $inc 0 98.18 $plane
set n4 [$ns node]; $n4 set-position $alt $inc 0 130.91 $plane
set n5 [$ns node]; $n5 set-position $alt $inc 0 163.64 $plane
set n6 [$ns node]; $n6 set-position $alt $inc 0 196.36 $plane
set n7 [$ns node]; $n7 set-position $alt $inc 0 229.09 $plane
set n8 [$ns node]; $n8 set-position $alt $inc 0 261.82 $plane
set n9 [$ns node]; $n9 set-position $alt $inc 0 294.55 $plane
set n10 [$ns node]; $n10 set-position $alt $inc 0 327.27 $plane
incr plane; #增加轨道面编号
……
#地面终端会不停的切换，以下设置旨在优化切换
$n0 set_next $n10; $n1 set_next $n0; $n2 set_next $n1; $n3 set_next $n2
$n4 set_next $n3; $n5 set_next $n4; $n6 set_next $n5; $n7 set_next $n6
$n8 set_next $n7; $n9 set_next $n8; $n10 set_next $n9
……
```

基于上述的场景，构建如下的实验脚本进行仿真分析。其中，纬度和经度信息请根据实际需要调整，具体如何查询纬度和经度详见本章的 6.2 节。

```
global ns
set ns [new Simulator]
#切换和路由相关的参数设置，具体解释参考[2]
HandoffManager/Term set elevation_mask_ 8.2
HandoffManager/Term set term_handoff_int_ 10
HandoffManager/Sat set sat_handoff_int_ 10
```

[2] http://vlabs.iitkgp.ernet.in/ant/4/theory/

```
HandoffManager/Sat set latitude_threshold_ 60
HandoffManager/Sat set longitude_threshold_ 10
HandoffManager set handoff_randomization_ true
SatRouteObject set metric_delay_ true
SatRouteObject set data_driven_computation_ true
ns-random 1
Agent set ttl_ 32
global opt
set opt(chan)           Channel/Sat
set opt(bw_down)        1.5Mb
set opt(bw_up)          1.5Mb
set opt(bw_isl)         25Mb
set opt(phy)            Phy/Sat
set opt(mac)            Mac/Sat
set opt(ifq)            Queue/DropTail
set opt(qlim)           50
set opt(ll)             LL/Sat
set opt(wiredRouting)   OFF
set opt(alt)            780; #极轨道高度，单位 Km
set opt(inc)            86.4;#倾斜角度
#设置输出文件
set outfile [open sat-iridium.tr w]
$ns trace-all $outfile
#节点参数配置
$ns node-config -satNodeType polar \
        -llType $opt(ll) \
        -ifqType $opt(ifq) \
        -ifqLen $opt(qlim) \
        -macType $opt(mac) \
        -phyType $opt(phy) \
        -channelType $opt(chan) \
        -downlinkBW $opt(bw_down) \
        -wiredRouting $opt(wiredRouting)
#导入卫星节点和链路配置
set alt $opt(alt)
set inc $opt(inc)
source sat-iridium-nodes.tcl
source sat-iridium-links.tcl
#以下代码配置数据传输的终端节点
$ns node-config -satNodeType terminal
set n100 [$ns node]
$n100 set-position 39.54 116.28; #纬度、经度设置，此处设置为北京
set n101 [$ns node]
$n101 set-position 31.12 121.26; #纬度、经度设置，此处设置为上海
```

```
$n100 add-gsl polar $opt(ll) $opt(ifq) $opt(qlim) $opt(mac) $opt(bw_up) \
   $opt(phy) [$n0 set downlink_] [$n0 set uplink_]
$n101 add-gsl polar $opt(ll) $opt(ifq) $opt(qlim) $opt(mac) $opt(bw_up) \
   $opt(phy) [$n0 set downlink_] [$n0 set uplink_]
#设置测试的数据流
$ns trace-all-satlinks $outfile
set udp0 [new Agent/UDP]
$ns attach-agent $n100 $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 attach-agent $udp0
$cbr0 set interval_ 60.01
set null0 [new Agent/Null]
$ns attach-agent $n101 $null0
$ns connect $udp0 $null0
$ns at 1.0 "$cbr0 start"
#计算路由
set satrouteobject_ [new SatRouteObject]
$satrouteobject_ compute_routes
$ns at 86400.0 "finish"
proc finish {} {
      global ns outfile
      $ns flush-trace
      close $outfile
      exit 0
}
$ns run
```

根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab6**”命令切换路径，接着使用“**touch sat-iridium.tcl**”创建脚本文件，将上述代码抄录至该文件中；

②在命令行终端运行“**ns sat-iridium.tcl**”执行脚本，成功后会产生 **sat-iridium.tr** 文件，数据记录格式如表 6.2。注意，在执行“**sat-iridium.tcl**”前需将“**sat-iridium-nodes.tcl**”和“**sat-iridium-links.tcl**”复制到“**/home/wiotlab/WIoTLab/exps/lab6**”中。

表 6.2　trace 文件记录格式

Event	Time	From Node	To Node	Pkt Type	Pkt Size	Flags	Fid	Src Addr	Dst Addr	Seq Num	Pkt ID	From Node Latitude	From Node Longitude	To Node Latitude	To Node Longitude

③分析星链上数据包端到端投递时延，具体脚本代码如下：

```
#星链上数据包端到端投递时延计算
BEGIN {
      highest_packet_id=0;
}
{
```

```
        action=$1;
        time=$2;
        from=$3;
        to=$4;
        type=$5;
        size=$6;
        flow_id=$8;
        src=$9;
        dst=$10;
        seq_no=$11;
        packet_id=$12;
        if (packet_id > highest_packet_id) highest_packet_id =packet_id;
        if (start_time[packet_id]==0) start_time[packet_id] =time;
        if(flow_id ==0&&action !="d") {
                if(action=="r"&&to=="67") {
                        end_time[packet_id]=time;
                } else {
                        end_time[packet_id]=-1;
                }
        }
}
END {
        for (packet_id=0;packet_id <= highest_packet_id;packet_id++) {
                start=start_time[packet_id];
                end=end_time[packet_id];
                delay=end-start;
                if (start < end) printf( "%f %f\n",start,delay)
        }
}
```

在命令行终端执行“**touch delay.awk**”生成空的 awk 脚本文件“**delay.awk**”，并将上述脚本代码抄录至该文件中。紧接着，执行“**awk -f delay.awk sat-iridium.tr > delay.d**”，会得到处理后的数据并保存在“**delay.d**”文件中。基于分析的数据，实现如下 gnuplot 绘图脚本：

```
set term eps
set out "delay.eps"
set size 1,0.7
set key right top
set ylabel "Delay(s)"
set xlabel "Time(s)"
plot "delay.d" u 1:2 w lp lw 2 lc rgb "black" t "E2E Delay"
```

在命令行终端执行“**touch delay.gp**”生成空的 gnuplot 绘图脚本文件“**delay.gp**”，并将上述代码抄录至该文件中。紧接着执行“**gnuplot delay.gp**”，会生成图 6.4 所示的结果，并保存

在文件“**delay.eps**”中。

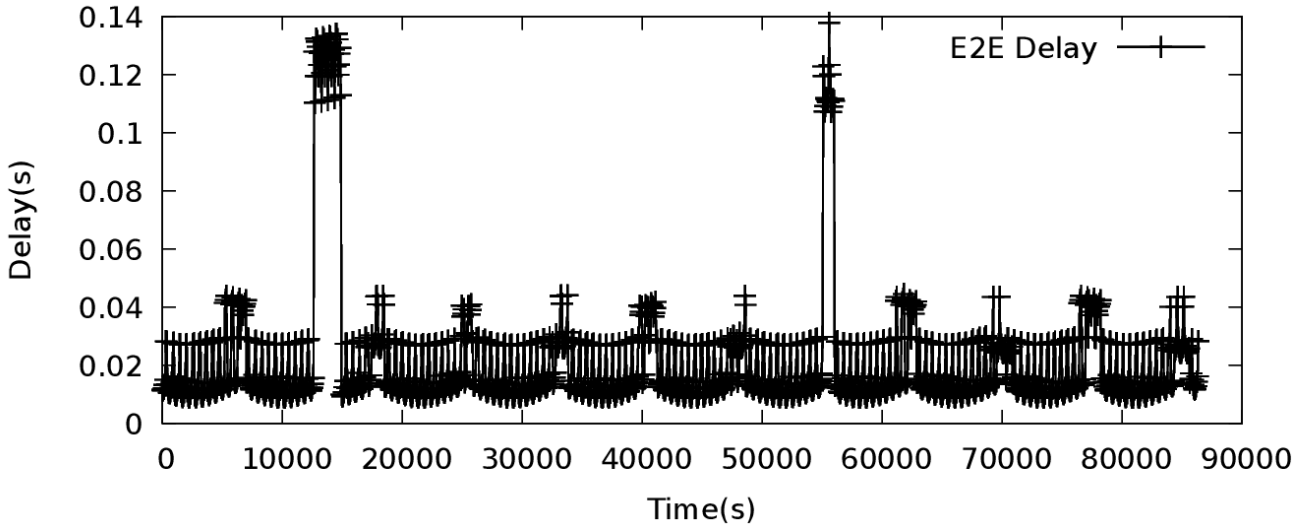


图 6.4　北京→上海(星链)端到端时延分析结果

5.3.2 Teledesic 仿真

该实验构建了包含 288 个节点、12 个轨道面的星座拓扑，同一个轨道面和跨轨道面的卫星链路设置如下代码所示(详细代码见“**ns-2.35/tcl/ex/sat-teledesic-links.tcl**”)。

```
for {set a 1} {$a <= 12} {incr a} {
	for {set i [expr $a * 100]} {$i < [expr $a * 100 + 24]} {incr i} {
		set imod [expr $i % 100]
		if {$imod == 23} {
			set next [expr $a * 100]
		} else {
			set next [expr $i + 1]
		}
		if {$imod == 23} {
			set next2 [expr $a * 100 + 1]
		} elseif {$imod == 22} {
			set next2 [expr $a * 100]
		} else {
			set next2 [expr $i + 2]
		}
		$ns add-isl intraplane $n($i) $n($next) $opt(bw_isl) $opt(ifq) $opt(qlim)
		$ns add-isl intraplane $n($i) $n($next2) $opt(bw_isl) $opt(ifq) $opt(qlim)
	}
}
for {set i 100} {$i < 124} {incr i} {
	set next [expr $i + 100]
	$ns add-isl interplane $n($i) $n($next) $opt(bw_isl) $opt(ifq) $opt(qlim)
}
```

```
for {set a 3} {$a <= 12} {incr a} {
	for {set i [expr $a * 100]} {$i < [expr $a * 100 + 24]} {incr i} {
		set prev [expr $i - 100]
		set prev2 [expr $i - 200]
		$ns add-isl interplane $n($i) $n($prev) $opt(bw_isl) $opt(ifq) $opt(qlim)
		$ns add-isl interplane $n($i) $n($prev2) $opt(bw_isl) $opt(ifq) $opt(qlim)
	}
}
for {set i 100} {$i < 112} {incr i} {
	set j [expr 1311 - $i]
	$ns add-isl crossseam $n($i) $n($j) $opt(bw_isl) $opt(ifq) $opt(qlim)
}
for {set i 112} {$i < 124} {incr i} {
	set j [expr 1335 - $i]
	$ns add-isl crossseam $n($i) $n($j) $opt(bw_isl) $opt(ifq) $opt(qlim)
}
```

卫星节点生成如下代码所示(详细代码见“**ns-2.35/tcl/ex/sat-teledesic-nodes.tcl**”)**:**

```
for {set a 1} {$a <= 12} {incr a} {
	for {set i [expr $a * 100]} {$i < [expr $a * 100 + 24]} {incr i} {
		if {$a % 2} {
			set stagger 0
		} else {
			set stagger 7.5
		}
		set imod [expr $i % 100]
		set n($i) [$ns node]
		$n($i) set-position $alt $inc [expr 15 * ($a - 1)] \
			[expr $stagger + $imod * 15] $a
	}
}
for {set a 1} {$a <= 12} {incr a} {
	for {set i [expr $a * 100]} {$i < [expr $a * 100 + 24]} {incr i} {
		if {$i % 100} {
			set next [expr $i - 1]
		} else {
			set next [expr $a * 100 + 23]
		}
		$n($i) set_next $n($next)
	}
}
```

基于上述的场景，构建如下的实验脚本进行仿真分析。其中，纬度和经度信息请根据实际需要调整，具体如何查询纬度和经度详见本章的 6.2 节。

```
Node/SatNode set time_advance_ 0
```

```
HandoffManager/Term set elevation_mask_ 40
HandoffManager/Term set term_handoff_int_ 10
HandoffManager/Sat set sat_handoff_int_ 10
HandoffManager/Sat set latitude_threshold_ 60
HandoffManager/Sat set longitude_threshold_ 8
HandoffManager set handoff_randomization_ true
SatRouteObject set metric_delay_ true
SatRouteObject set data_driven_computation_ true
ns-random 1
Agent set ttl_ 32
global opt
set opt(chan)                Channel/Sat
set opt(bw_down)             1.5Mb
set opt(bw_up)               1.5Mb
set opt(bw_isl)              155Mb
set opt(phy)                 Phy/Sat
set opt(mac)                 Mac/Sat
set opt(ifq)                 Queue/DropTail
set opt(qlim)                50
set opt(ll)                  LL/Sat
set opt(wiredRouting)        OFF
set opt(alt)                 1375
set opt(inc)                 84.7
global ns
set ns [new Simulator]
set outfile [open sat-teledesic.tr w]
$ns trace-all $outfile
$ns node-config -satNodeType polar \
        -llType $opt(ll) \
        -ifqType $opt(ifq) \
        -ifqLen $opt(qlim) \
        -macType $opt(mac) \
        -phyType $opt(phy) \
        -channelType $opt(chan) \
        -downlinkBW $opt(bw_down) \
        -wiredRouting $opt(wiredRouting)

set alt $opt(alt)
set inc $opt(inc)
source sat-teledesic-nodes.tcl
source sat-teledesic-links.tcl
$ns node-config -satNodeType terminal
set n100 [$ns node]
$n100 set-position 39.54 116.28
```

```
set n101 [$ns node]
$n101 set-position 31.12 121.26
$n100 add-gsl polar $opt(ll) $opt(ifq) $opt(qlim) $opt(mac) $opt(bw_up) \
   $opt(phy) [$n(100) set downlink_] [$n(100) set uplink_]
$n101 add-gsl polar $opt(ll) $opt(ifq) $opt(qlim) $opt(mac) $opt(bw_up) \
   $opt(phy) [$n(100) set downlink_] [$n(100) set uplink_]
$ns trace-all-satlinks $outfile

set udp0 [new Agent/UDP]
$ns attach-agent $n100 $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 attach-agent $udp0
$cbr0 set interval_ 60
set null0 [new Agent/Null]
$ns attach-agent $n101 $null0
$ns connect $udp0 $null0
$ns at 1.0 "$cbr0 start"

set satrouteobject_ [new SatRouteObject]
$satrouteobject_ suppress_initial_computation
$ns at 0.5 "$satrouteobject_ compute_routes"
$ns at 86400.0 "finish"
proc finish {} {
      global ns outfile
      $ns flush-trace
      close $outfile
      exit 0
}
$ns run
```

根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab6**”命令切换路径，接着使用“**touch sat-teledesic.tcl**”创建脚本文件，将上述代码抄录至该文件中；

②在命令行终端运行“**ns sat-teledesic.tcl**”执行脚本，成功后会产生 **sat-teledesic.tr** 文件，格式同前。注意，在执行“**sat-teledesic.tcl**”前需将“**sat-teledesic-nodes.tcl**”和“**sat-teledesic-links.tcl**”复制到“**/home/wiotlab/WIoTLab/exps/lab6**”中。

③分析星链上数据包端到端投递时延，具体脚本代码如前述“**delay.awk**”(仅需修改“**to=="289"**”)。执行“**awk -f delay.awk sat-teledesic.tr > delay.d**”，会得到处理后的数据并保存在“**delay.d**”文件中。紧接着执行“**gnuplot delay.gp**”，会生成图 6.5 所示的结果，并保存在文件“**delay.eps**”中。

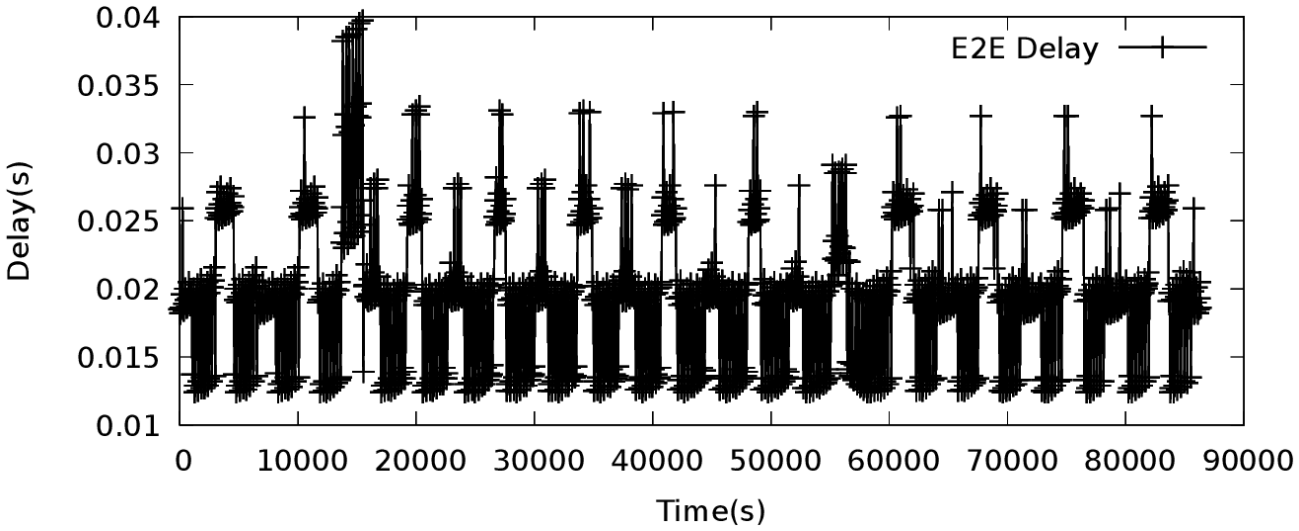


图 6.5　北京→上海(星链)端到端时延分析结果

6.4 扩展实验*

本章介绍了低轨卫星网络的发展及相关星座，并用 NS-2 仿真分析了 Iridium 和 Teledesic 两个经典卫星系统的端到端时延性能。在此基础上，读者可深入探索，扩展实验，具体如下：

(1)增加分析指标，比如数据传输抖动、丢包等，能够更加全面的了解不同卫星星座的性能差异；

(2)查询不同地理位置的纬度和经度数据，重新设置实验脚本中的信息，并重复 6.3 节的过程，了解不同距离对于卫星传输的影响。

参考文献

[1] W. Jiang; Y. Zhan; X. Xiao, et al. Network Simulators for Satellite-Terrestrial Integrated Networks: A Survey. IEEE Access, 2023, 11: 98269-98292.

[2] S. Kassing, D. Bhattacherjee, A. B. Águas, et al. Exploring the “Internet from space” with Hypatia. In Proc. IMC, Oct. 2020, Virtual Event, USA.

[3] X. Cao, X. Zhang. SaTCP：Link-Layer Informed TCP Adaptation for Highly Dynamic LEO Satellite Networks. In Proc. INFOCOM, May 2023, New York, USA.